

Do Voting Rules Reduce the Damage from Malicious AI Agents? A Simulation Study

Authors

Rong Jin <rongjin@kth.se>

Hieu Vu <hieuvtm@kth.se>

Abstract

Multi-agent LLM systems can contain a malicious insider: an agent that participates normally while covertly sabotaging the shared objective. This study asks whether the team's own collective decision rule limits the damage it does. We will run a controlled experiment in the open-source Among Us sandbox, where impostors deceive in pursuit of the game objective rather than on instruction. A 2×2 between-games design manipulates the vote threshold for ejection (plurality versus absolute majority) and the discussion protocol preceding it (free-form versus mandatory justification). With at least twenty games per condition we analyse win rate and wrongful removal by logistic regression, damage by count models, and detection time by survival analysis. The contribution is evidence on whether governance is a usable black-box lever for multi-agent safety.

1 Introduction

1.1 Background and Rationale

LLM agents are increasingly deployed in teams that divide a shared task and act on a common artefact such as a codebase or a plan. Their safety then depends on how the team behaves when one member does not share its goal. We study the malicious insider: an agent that participates normally while covertly sabotaging the objective. It needs no deliberate attacker, since deception can emerge from a model pursuing a goal [3], [4], and a compromised message can propagate adversarial instructions across a team [10].

Two defences exist. The first inspects the suspect agent through probes on internal activations, presupposing access a deployed team often lacks. The second changes the rules by which the team acts: what evidence must precede an accusation, and how many votes remove a member. We call this governance – cheap, model-agnostic, treating every agent as a black box. Every such rule cuts both ways. A permissive rule ejects the insider early but makes wrongful ejection easy; a strict rule protects innocents but leaves the insider in place

longer; mandatory justification may raise evidence quality or merely give a deceiver a better stage. Which effect dominates cannot be settled by reasoning alone, yet an engineer configuring a multi-agent framework today must choose with no evidence base.

1.2 Theory and Related Work

Prior work. Distributed systems treat a decision threshold as a safety parameter: agreement holds only when faulty participants are bounded [1], [11]. That assumption does not transfer, since honest LLM agents are fallible reasoners whose votes are emergent judgements. Voting theory also predicts the threshold should matter, but sincere voting is not an equilibrium once voters reason strategically [9]. On deception, [3] and [4] show deception instrumental to a goal rather than instructed, and [2] carries this into the Among Us sandbox, extending AmongAgents [5] and finding deception easier to produce than to detect by white-box probes. AI control asks which protocol limits damage from a subversive model [6], but is hierarchical rather than peer-to-peer, and deliberation work [7], [8] assumes honest participants.

The gap. For a team with an undisclosed adversarial member and only black-box access, the collective decision procedure has not been treated as a manipulable independent variable, nor its effect measured against damage rather than detection. The sandbox closest to our setting [2] holds the voting phase fixed and excludes games in which the insider is never identified.

1.3 Research Problem, Aim, and Research Questions

Problem and aim. In a multi-agent LLM system with an undisclosed insider, the good agents know an insider may exist but not who it is. Existing work asks whether such deception can be detected, not which governance mechanism minimises the damage it inflicts. Within one fixed sandbox we measure how the removal threshold and the form of the discussion affect the damage caused by the insider and the rate of wrongful removal, holding model, task, team size and insider count constant.

Objectives. O1: by week 1, reproduce the sandbox and run three complete games with usable logs. O2: by week 2, implement a log parser extracting every dependent variable, validated at $\geq 95\%$ field agreement against ten annotated transcripts. O3: by week 3, implement both governance factors as configuration options, with unit tests on the tally logic. O4: by week 3, pilot five games per condition and fix the run count and analysis plan. O5: by week 5, archive

≥ 20 valid games per condition. O6: by week 6, produce all pre-specified comparisons and five figures from one script.

Research questions. RQ1 (confirmatory): with one insider, how does the vote threshold (plurality versus an absolute majority of living agents) affect (a) the good-team win rate and (b) the damage absorbed? RQ2 (confirmatory): how does requiring a structured justification – an accusation with supporting evidence – before voting affect the rate of wrongful removal? RQ3 (secondary): how does each mechanism affect detection speed, measured as meetings until the insider is first correctly ejected, with never-ejected games right-censored?

Hypotheses. H1: an absolute majority reduces wrongful removal but increases damage absorbed, so the effect on the win rate is non-monotone. H2: requiring a justification lowers the wrongful-removal rate. H3 (exploratory): the factors interact, the benefit of justification being larger under plurality.

Scope. We manipulate governance only: one insider, constant team composition and model, no detection from model internals.

2 Research Method

2.1 Research Design and Methodological Rationale

Overall design. We will run a controlled, quantitative experiment in an agent-based simulation: LLM agents play the game out turn by turn and we observe outcomes. The design is a 2×2 between-games factorial, with vote threshold and discussion protocol as independent variables. The unit of analysis is one complete game; because outcomes are stochastic, each condition is replicated many times. Everything else – model, decoding parameters, team size, insider count, map and task set, discussion rounds, step limit and all unmanipulated prompt text – is held fixed. The factorial design yields both main effects and tests H3.

Alternatives and assumptions. A game-theoretic treatment would have to fix the insider's strategy in advance, assuming away the emergent deception that makes the problem hard; qualitative analysis alone could not support a causal claim. We assume in-game destructive actions proxy damage to a shared objective. With one sandbox and one model, external validity is argued by analogy.

2.2 Data, Materials, Participants, or Other Sources

Simulation environment. The core material is the open-source Among Us sandbox released with [2], a public repository under CC0 1.0 [12]. It provides game logic, orchestration, agent prompting, per-game logging and evaluation scripts. Impostors map onto our insiders, crewmates onto the good agents. Play alternates between a task phase, in which the impostor fakes tasks and takes destructive actions, and a meeting phase, in which survivors discuss and vote.

Team configuration. The published configuration uses seven agents with two impostors; we use five with one insider. A single insider matches the threat model and avoids insider coordination as an uncontrolled factor. Five is the smallest team in which the thresholds can diverge after the first kill: divergence needs four living voters, since with three – and a skip vote counted as a ballot option (Table 1) – plurality already requires a majority. As a manipulation check we report the share of meetings in which re-tallying under the other threshold would change the decision. If week 1 shows this configuration is unsupported, we retain the published setup and report the changed prevalence as a limitation.

Model selection. A single commercial model, accessed exclusively through its provider's API, is held constant across conditions; no model weights are self-hosted. The shortlist comprises GLM and DeepSeek, chosen in the pilot against four criteria: cost per game within budget; latency low enough to finish in two weeks; coherent play ($\geq 95\%$ of votes and actions parsing into valid moves); and a baseline win rate inside $[0.15, 0.85]$.

Units and sampling. One completed game is one experimental unit. Roles are randomised per game and each seed recorded. Conditions are interleaved rather than blocked, so drift in the hosted model affects all four equally. A game is excluded and regenerated if it aborts on an API error or enters a malformed-output loop.

Table 1. Variables in the experiment.

Role	Variable	Definition and levels
Independent	Vote threshold	Plurality: most-voted agent ejected, ties resolved as no ejection. Absolute majority: more than half the living voters.
Independent	Discussion protocol	Free-form: the default discussion prompt. Justification: an accusation (named agent or abstention) plus one item of evidence.

Role	Variable	Definition and levels
Dependent	Good-team win	Binary; the game ends in a crew win.
Dependent	Damage absorbed	Count of the insider's successful destructive actions before the game ends.
Dependent	Wrongful removal	Binary per game (at least one good agent ejected) and count per game.
Dependent	Detection time	Meeting index at which the insider is first ejected; right-censored at game end.
Controlled	Fixed factors	Model, decoding parameters, team size, insider count, map and task set, discussion rounds, step limit, prompt text.
Recorded	Run metadata	Condition, seed, model identifier, timestamp, duration, token counts and cost.

Budget and ethics. Inference is bought through a hosted API and runs are parallelised on ordinary laptops. The pilot measures cost and latency, and the run count is set so spend stays inside a budget agreed with the mentor; if it cannot, Section 3.3 reduces the design rather than the replications. There are no human participants and no personal data, and we develop no new attack technique.

2.3 Work Procedure and Data Collection

Steps 1–2 – Reproduce and instrument (weeks 1–2). Clone the repository [12], install pinned dependencies, and run at least three complete games with the published configuration, recording the destructive actions available, the default voting code, whether team size is parameterised, and which fields the logs contain. Gate G1: if a full game cannot be completed by the end of week 1, we invoke risk R1. We then build a parser turning raw logs into three tables – per game, per meeting, per action. Gate G2: $\geq 95\%$ field agreement with ten hand-annotated transcripts.

Step 3 – Implement the conditions (weeks 2–3). Both factors are configuration flags, not code edits. The threshold factor changes only the tally function, covered by unit tests on synthetic ballots checking ties, abstentions and the shrinking electorate. The discussion factor changes only the instruction text before the final discussion round, plus a format check that re-prompts once if a statement has no parseable accusation and evidence. The free-form condition receives a length-matched neutral instruction, so prompt length is not confounded.

Steps 4–5 – Pilot and freeze the plan (week 3). Run five games per condition. The pilot delivers the model choice, variance estimates, and cost and time per game. From these we set runs per condition: a floor of 15, a planned 20, a target of 30. Gate G3: sanity checks pass and projected cost is inside budget. Before any main-experiment game is run, the analysis script, test list, exclusion criteria and damage weights are committed and tagged; anything computed afterwards is exploratory.

Steps 6–8 – Run, analyse and package (weeks 4–6). All four conditions run interleaved, one recorded seed per game, with automatic retry on transient API failure. The frozen script then produces the tests and figures in Section 2.4. In parallel, a qualitative sub-study explains mechanism: twenty meetings ending in the ejection of a good agent are sampled across conditions and coded independently by both authors. On completion we tag a reproducibility package with dependencies, seeds, logs, parsed tables, the script and a README.

2.4 Data Analysis and Quality Assurance

Operational definitions. Per game we record: the outcome, a good-team win or not; damage absorbed, a count of the insider's successful destructive actions, combined into a pre-specified equally weighted composite fixed before the pilot; wrongful removal, true if at least one good agent is ejected; detection time, the meeting index at which the insider is first ejected, right-censored at game end; and exposure, the task rounds the insider survives, used as the count-model offset.

Confirmatory analysis. All confirmatory tests are two-sided at $\alpha = 0.05$ and Holm-corrected; effect sizes and 95% confidence intervals are reported for every comparison and are what we interpret. Binary outcomes use logistic regression with both factors and their interaction. Damage uses negative binomial regression with a log-exposure offset. Detection time uses Kaplan–Meier and a Cox model; censoring never-detected games is deliberate, since conditioning on correct ejections would select the games in which governance worked.

Table 2. Mapping from research questions to metrics, tests and planned figures.

Question	Metric	Planned test	Figure
RQ1a	Good-team win rate	Logistic regression (threshold $\times$ protocol); Fisher's exact per cell	F1: win rate by condition with 95% CI
RQ1b	Damage absorbed	Negative binomial GLM with log-exposure offset; Mann–Whitney with Cliff's delta	F2: damage by condition; F3: cumulative damage against round

Question	Metric	Planned test	Figure
RQ2	Wrongful-removal rate	Logistic regression; two-proportion test with risk difference and 95% CI	F4: wrongful-removal rate by condition
RQ3	Meetings to first correct ejection	Kaplan–Meier with log-rank test; Cox proportional hazards	F5: Kaplan–Meier detection curves by condition
H3	All primary outcomes	Interaction term in the regressions above	F1, F4, grouped by both factors

Decision rules. H1 is supported if the threshold effect gives fewer wrongful removals and more damage under absolute majority while the win-rate interval includes zero or reverses sign across the other factor; it is contradicted if the win rate rises monotonically with the threshold. H2 is supported if the justification coefficient gives an odds ratio below one with an interval excluding one, H3 if the interaction term is negative with an interval excluding zero.

Statistical power. At 20 games per condition a main-effect comparison pools 40 games per level and detects a win-rate difference of about 29 percentage points with 80% power at $\alpha = 0.05$ against a baseline of 0.5; at 30 games it detects about 25, and a single cell-to-cell comparison about 33. The study is powered for large effects only, so a non-significant comparison is inconclusive rather than evidence of no effect.

Threats to validity. Internally, the manipulated prompt differs in length as well as content, addressed by the length-matched filler, and the hosted model may change during the runs, addressed by interleaving conditions and recording the model identifier per game. Destructive actions proxy real harm and the composite depends on chosen weights, so component counts are reported separately. All findings come from one sandbox and model, so conclusions concern a mechanism rather than deployment.

Quality assurance. Randomised roles and interleaved conditions guard against assignment effects, and replication is the central control on stochasticity. Because LLM sampling means a run cannot be reproduced token for token even at fixed seed, we treat the raw logs as the primary record: the analysis is reproducible from archived logs even where the generation is not. Qualitative codes are double-coded with Cohen's κ reported.

2.5 Use of AI Tools

We distinguish the LLM that is the object of study – whose outputs are data, never claims – from the assistants we use as instruments. Coding assistants will draft the parser, tests and plotting code, but no generated code is committed unread: every change is reviewed line by line by the author who did not prompt for it. LLM-based search triages papers, but no source enters the reference list unless an author has read it. AI tools improve grammar and clarity only. Both authors take full responsibility for the intellectual content.

3 Project Planning and Feasibility

3.1 Work Tasks and Milestones

The work breaks into eight tasks. T1–T6 form the critical path and proceed in sequence; T7 follows the main runs and T8 runs alongside. Each ends in an objectively checkable milestone.

Table 3. Work tasks, dependencies and milestones. Weeks are relative to the mentor's approval of this plan.

Task	Week	Depends on	Milestone
T1 Reproduce sandbox; audit logs and configuration	W1	–	M1: three games end-to-end; log schema documented (G1)
T2 Build and validate the log parser	W2	T1	M2: $\geq 95\%$ field agreement on ten transcripts (G2)
T3 Implement the two governance factors	W2–W3	T1	M3: four conditions selectable by flag; tally tests pass
T4 Pilot, model choice and sample size	W3	T2, T3	M4: five games per condition; model and run count fixed (G3)
T5 Freeze the analysis plan	W3	T4	M5: script, tests, exclusions and weights tagged
T6 Run the main experiment	W4–W5	T5	M6: ≥ 20 valid games per condition archived with seeds
T7 Analysis, figures and qualitative sub-study	W5–W6	T6	M7: all confirmatory tests and five figures from the script
T8 Report and reproducibility package	W1–W6	–	M8: report submitted; repository tagged with logs and instructions

Schedule and buffer. The plan front-loads risk: the two decisions that could force a change of direction – whether the environment runs at all, and whether the experiment is affordable – fall at the end of weeks 1 and 3. The main experiment has two weeks although the pilot

estimate should place it within one. If T6 has not reached fifteen valid games per condition by mid-week 5, we drop to the three-condition design rather than reduce replications.

3.2 Allocation of Responsibilities

The group works as a pair rather than splitting the project into two halves. Both authors work on every task; what rotates is who leads and who reviews. Rong leads T1 and T3 with Hieu reviewing, Hieu leads T2 and T7 with Rong reviewing, and T4–T6 and T8 are joint: the main runs are split across both machines, and both authors code the sampled meetings, as Cohen's κ requires two coders.

Progress monitoring. All work lives in one repository with a branch per task; nothing reaches main without a review from the other author. We hold a 30-minute planning meeting each Monday and a 30-minute sync each Friday, checked against the milestones in Table 3. Mentor meetings are planned for week 2 and week 4.

3.3 Implementational Risks and Mitigation Strategies

Table 4 states, for each of the four dominant risks, how we reduce its chance and what we do if it happens anyway.

Table 4. Principal implementation risks, prevention and contingency.

Risk	Likelihood / impact	Prevention	Backup plan
R1. Sandbox cannot be run, or the five-agent configuration is unsupported	Medium / high	Attempt reproduction in week 1 before anything else, with gate G1 as a stop.	Retain the published seven-agent, two-impostor setup, which leaves the governance contrast intact; else fall back to AmongAgents [5].
R2. Inference cost or latency caps the achievable number of games	Medium / high	Estimate cost and latency from the pilot before the main runs (G3); choose the smallest coherent model.	Drop to a three-condition design, removing a quarter of the runs and the interaction test; else run at the floor of fifteen.
R3. Default logs omit events needed for the dependent variables	Medium / medium	Audit the log schema against each dependent variable in week 1, before any experimental runs.	Add explicit event logging for destructive actions, ejections and wins; else reconstruct from transcripts with an audited parser.
R4. The chosen model plays incoherently	Medium / high	Apply the pilot sanity checks: parseable votes and actions, genuine win conditions, baseline win rate away from 0 and 1.	Move to the next model on the shortlist and repeat the pilot; if none is affordable, reduce the number of conditions.

A fifth possibility deserves naming: the governance rules may make no measurable difference at this sample size. We have designed for that outcome rather than against it, by pre-specifying the analysis and retaining the qualitative sub-study.

References

- [1] L. Lamport, R. Shostak, and M. Pease, “The Byzantine Generals Problem,” *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, Jul. 1982.
- [2] S. Golechha and A. Garriga-Alonso, “Among Us: A Sandbox for Measuring and Detecting Agentic Deception,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2504.04072.
- [3] E. Hubinger, C. Denison, J. Mu, M. Lambert, M. Tong, M. MacDiarmid, T. Lanham, D. M. Ziegler, T. Maxwell, N. Cheng, et al., “Sleepers Agents: Training Deceptive LLMs that Persist Through Safety Training,” arXiv:2401.05566, 2024.
- [4] R. Greenblatt, C. Denison, B. Wright, F. Roger, M. MacDiarmid, S. Marks, J. Treutlein, T. Belonax, J. Chen, D. Duvenaud, et al., “Alignment Faking in Large Language Models,” arXiv:2412.14093, 2024.
- [5] Y. Chi, L. Mao, and Z. Tang, “AmongAgents: Evaluating Large Language Models in the Interactive Text-Based Social Deduction Game,” arXiv:2407.16521, 2024.
- [6] R. Greenblatt, B. Shlegeris, K. Sachan, and F. Roger, “AI Control: Improving Safety Despite Intentional Subversion,” in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, PMLR vol. 235, 2024.
- [7] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, “Improving Factuality and Reasoning in Language Models through Multiagent Debate,” in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, PMLR vol. 235, 2024.
- [8] Y. Xu, S. Wang, P. Li, F. Luo, X. Wang, W. Liu, and Y. Liu, “Exploring Large Language Models for Communication Games: An Empirical Study on Werewolf,” arXiv:2309.04658, 2023.
- [9] D. Austen-Smith and J. S. Banks, “Information Aggregation, Rationality, and the Condorcet Jury Theorem,” *American Political Science Review*, vol. 90, no. 1, pp. 34–45, 1996.
- [10] D. Lee and M. Tiwari, “Prompt Infection: LLM-to-LLM Prompt Injection within Multi-Agent Systems,” arXiv:2410.07283, 2024.
- [11] M. Castro and B. Liskov, “Practical Byzantine Fault Tolerance,” in *Proceedings of the 3rd USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, New Orleans, LA, USA, 1999, pp. 173–186.
- [12] S. Golechha and A. Garriga-Alonso, “AmongUs: sandbox, game logs and probes,” GitHub repository, CC0 1.0. [Online]. Available: <https://github.com/7vik/AmongUs>. Accessed: Sep. 2026.

Appendices

Appendix A. Condition specification. The exact prompt text for each condition, the tally pseudocode for both thresholds, and the codebook for the qualitative sub-study will be fixed at the end of week 3 and included in the final report.